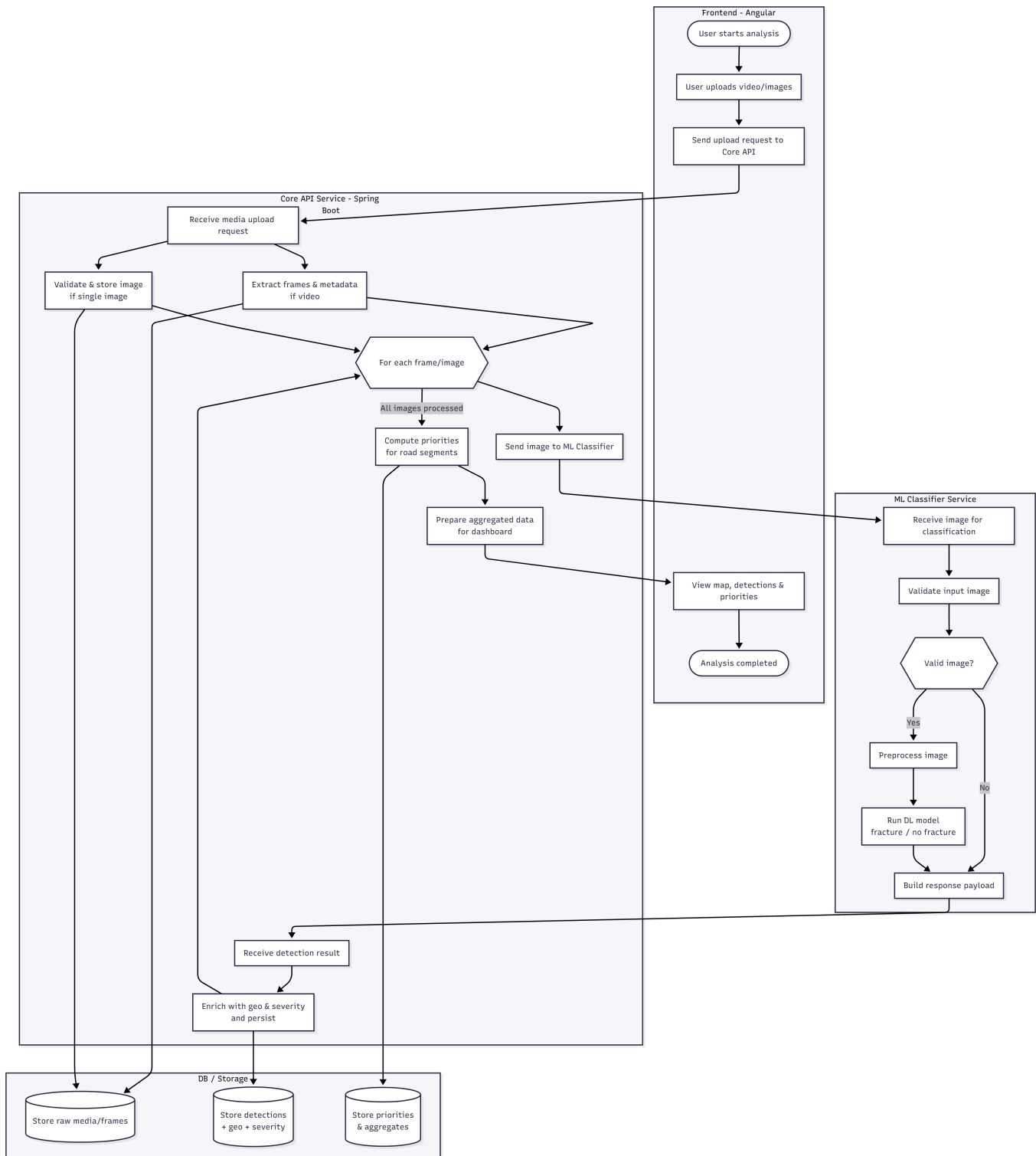


Compte Rendu - RoadSens-MS

Schéma d'architecture (vue d'ensemble)

BPMN (Business Process Modelling Notation):



- **Frontend Angular (RoadSense UI)**
 - Consomme uniquement les API REST de `core-api-service`.
- **Microservice 1 – core-api-service (Spring Boot)**
 - Point d'entrée backend (via API Gateway intégré ou contrôleurs REST).
 - Orchestration métier : upload média, appel au MS de ML, géoréférencement, calcul de gravité et priorisation, agrégations pour le dashboard.
- **Microservice 2 – reporting-service (Spring Boot, optionnel)**
 - Focalisé sur les lectures complexes : statistiques, exports GeoJSON/WMS, rapports; peut être fusionné avec `core-api-service` si tu veux rester à 2 MS.
- **Microservice 3 – ml-classifier-service (ML/DL)**
 - Service stateless d'inférence : reçoit une image, renvoie fracture / pas fracture + score + éventuellement bounding boxes.
- **Base(s) de données**
 - PostgreSQL/PostGIS pour les entités routières, détections, scores, priorités.
 - MinIO ou volume de fichiers pour stocker images/vidéos.
- **Communication**
 - Synchrone HTTP REST entre Angular → core-api → ml-classifier.
 - Accès DB direct uniquement depuis les MS backend (pas depuis Angular).



Rôle de chaque microservice

core-api-service (Spring Boot)

- Gérer l'upload de vidéos/images.
- Extraire les images (ou travailler directement image par image au début).
- Appeler `ml-classifier-service` pour obtenir le diagnostic sur chaque image.
- Associer chaque détection à une position (via PostGIS, OSRM, etc.).
- Calculer la gravité et la priorité des tronçons.
- Exposer des endpoints REST pour le dashboard (liste des défauts, carte, priorités, filtres).

reporting-service (Spring Boot) – si tu le gardes

- Construire des vues agrégées à partir de la base (groupement par région, route, période).
- Gérer les exports SIG (GeoJSON; WMS/WFS éventuellement via intégration GeoServer).
- Servir des endpoints dédiés “read-only / reporting” pour ne pas charger `core-api-service`.

ml-classifier-service (ML/DL)

- Charger le modèle de classification/sélection (CNN/YOLO, etc.).
- Offrir une API `POST /classify` qui:
 - Reçoit une image (multipart ou base64).
 - Retourne fracture / non fracture, score, et éventuellement bounding boxes.
- Rester stateless pour être facilement scalable.



Technologies par microservice

Microservice	Tech principales	DB associée
core-api-service	Spring Boot, Spring Web, Spring Data JPA, PostGIS driver	PostgreSQL/PostGIS + MinIO/fichiers
reporting-service	Spring Boot, Spring Data, projection/aggregation	PostgreSQL/PostGIS (en lecture)
ml-classifier-service	Python (FastAPI/Flask) + PyTorch/TensorFlow, CUDA si GPU	Pas de DB, seulement modèle + logs



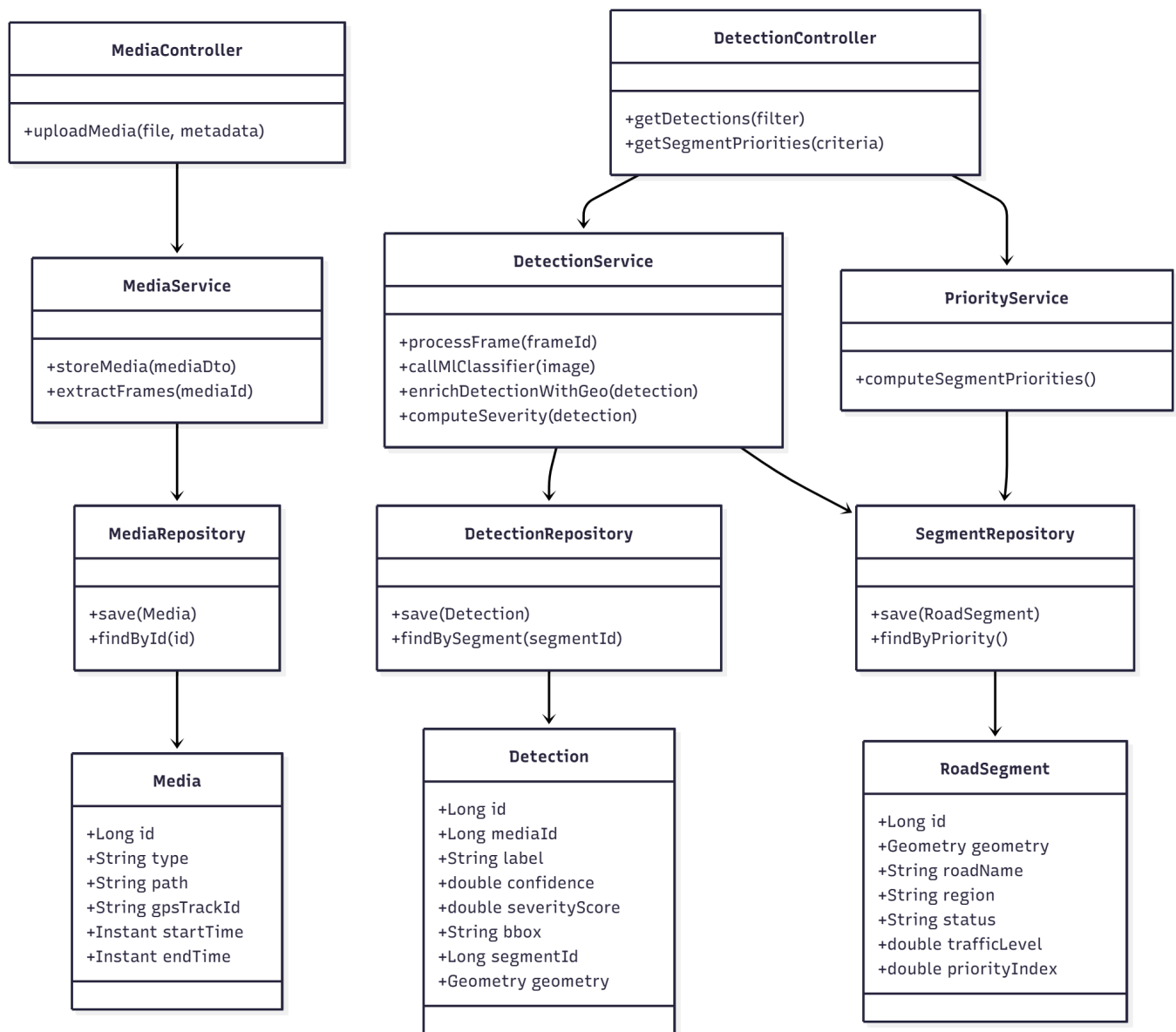
Méthodes de communication

- **Angular** → **core-api-service**
 - HTTP REST/JSON, endpoints typiques :
 - `POST /media/upload` (vidéo ou image)
 - `GET /detections` , `GET /segments/priorities` , `GET /stats/...`
- **core-api-service** → **ml-classifier-service**
 - HTTP REST synchrone:
 - `POST http://ml-classifier-service:PORT/classify`
 - Corps: image + éventuelles métadonnées.
 - Réponse: JSON avec label + score + boxes.
- **core-api-service** → **DB/Storage**
 - JDBC (Spring Data) vers PostgreSQL/PostGIS.
 - SDK S3 (MinIO) ou simple file system mount pour les médias.

»

Conception de chaque microservice

core-api-service – Diagramme de classes (texte)



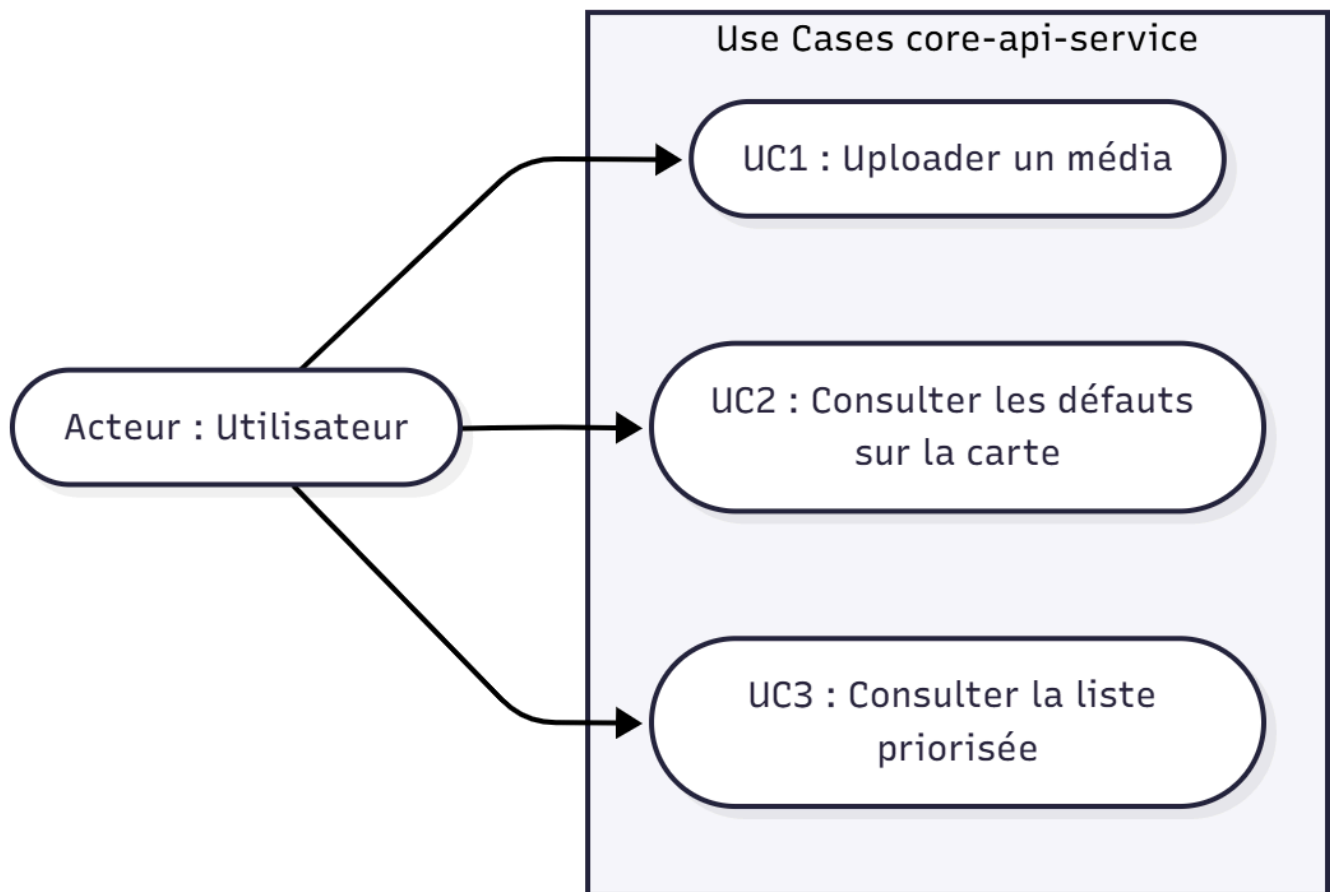
- **MediaController**
 - `uploadMedia(file, metadata)`
- **DetectionController**
 - `getDetections(filter)`
 - `getSegmentPriorities(criteria)`
- **MediaService**
 - `storeMedia(...)`
 - `extractFrames(...)`
- **DetectionService**
 - `processFrame(frameId)`
 - `callMLClassifier(image)`
 - `enrichDetectionWithGeo(...)`
 - `computeSeverity(...)`
- **PriorityService**

- `computeSegmentPriorities()`
- `MediaRepository` (JPA)
- `DetectionRepository` (JPA)
- `SegmentRepository` (JPA)

Entités principales :

- `Media` (id, type, path, gpsTrackId, timestampRange...)
- `Detection` (id, mediaId, bbox, label, confidence, severityScore, segmentId, geometry)
- `RoadSegment` (id, geometry, trafficLevel, priorityIndex, status...)

core-api-service – Cas d'utilisation

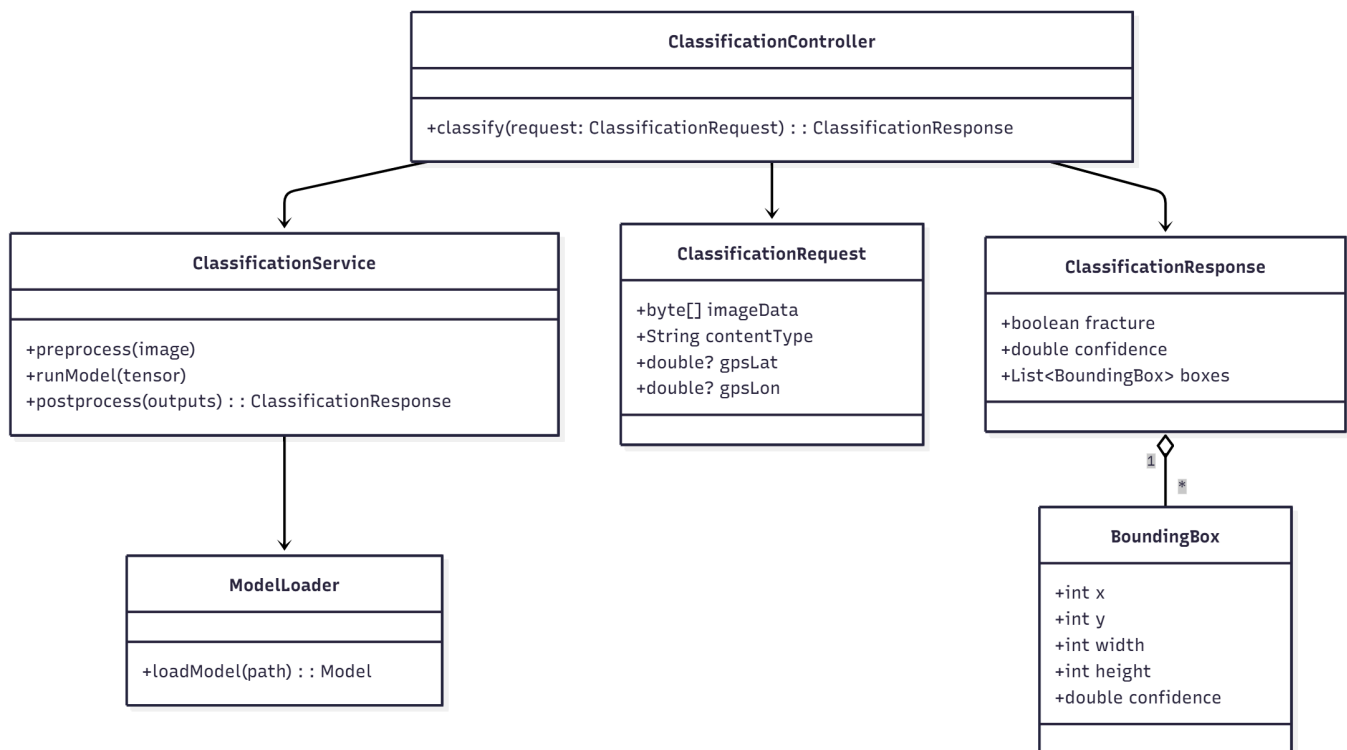


- UC1: **Uploader un média**
 - Acteur: Inspecteur / opérateur.
 - Scénario: L'utilisateur charge une vidéo ou une image → le service stocke le média, déclenche l'analyse image par image, puis met à jour la base avec les détections et scores.
- UC2: **Consulter les défauts sur la carte**

- Acteur: Responsable maintenance.
- Scénario: L'utilisateur ouvre le dashboard cartographique → le service renvoie les défauts filtrés (route, date, gravité) avec géométrie.
- UC3: **Consulter la liste priorisée des interventions**
 - Acteur: Responsable maintenance.
 - Scénario: L'utilisateur demande les tronçons les plus critiques → le service calcule/renvoie une liste triée par indice de priorité.

☞

ml-classifier-service – Diagramme de classes (texte)

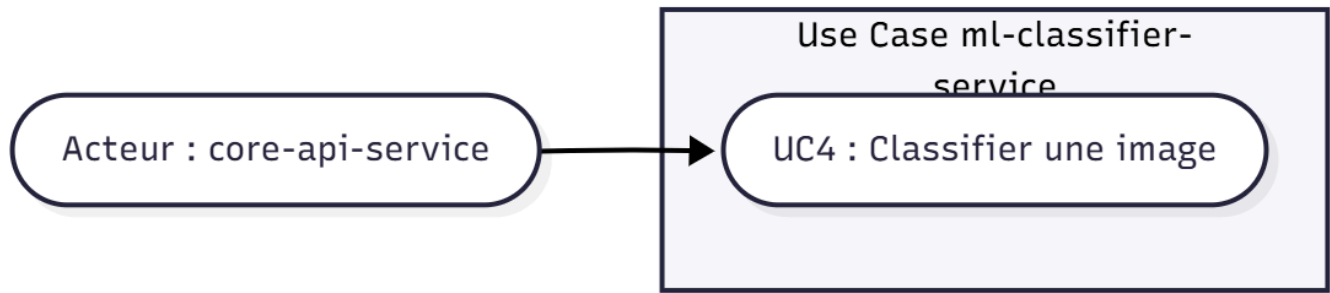


- **ClassificationController**
 - `classify(imageRequest)`
- **ClassificationService**
 - `preprocess(image)`
 - `runModel(imageTensor)`
 - `postprocess(outputs)`
- **ModelLoader**
 - `loadModel(modelPath)`

DTOs:

- `ClassificationRequest` (image, meta)
- `ClassificationResponse` (fracture:boolean, confidence:double, boxes:list)

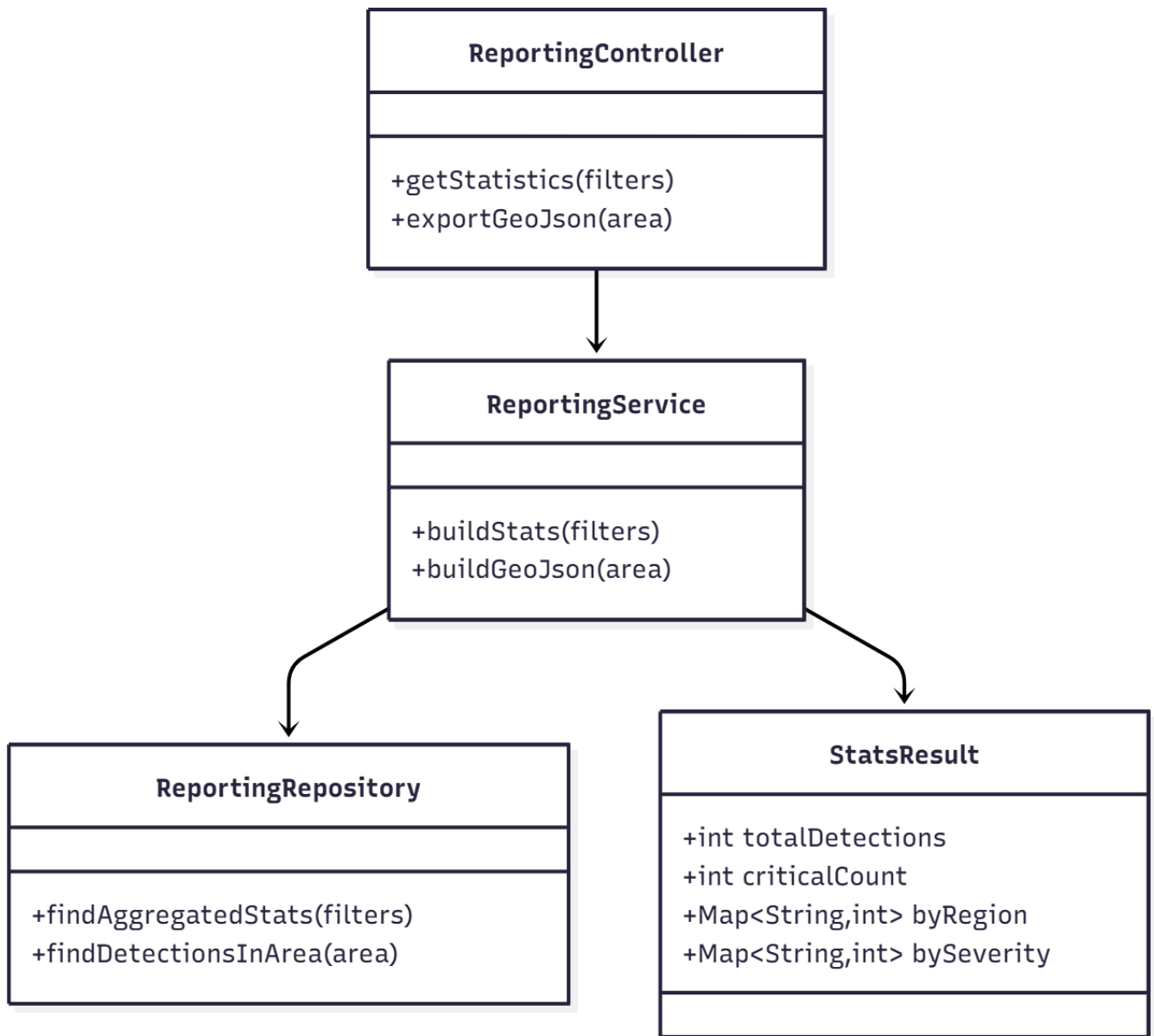
ml-classifier-service – Cas d'utilisation



- UC4: **Classifier une image**
 - Acteur: core-api-service.
 - Scénario: Le MS reçoit une image, exécute le modèle, renvoie fracture / pas fracture + score (et éventuellement bounding boxes).

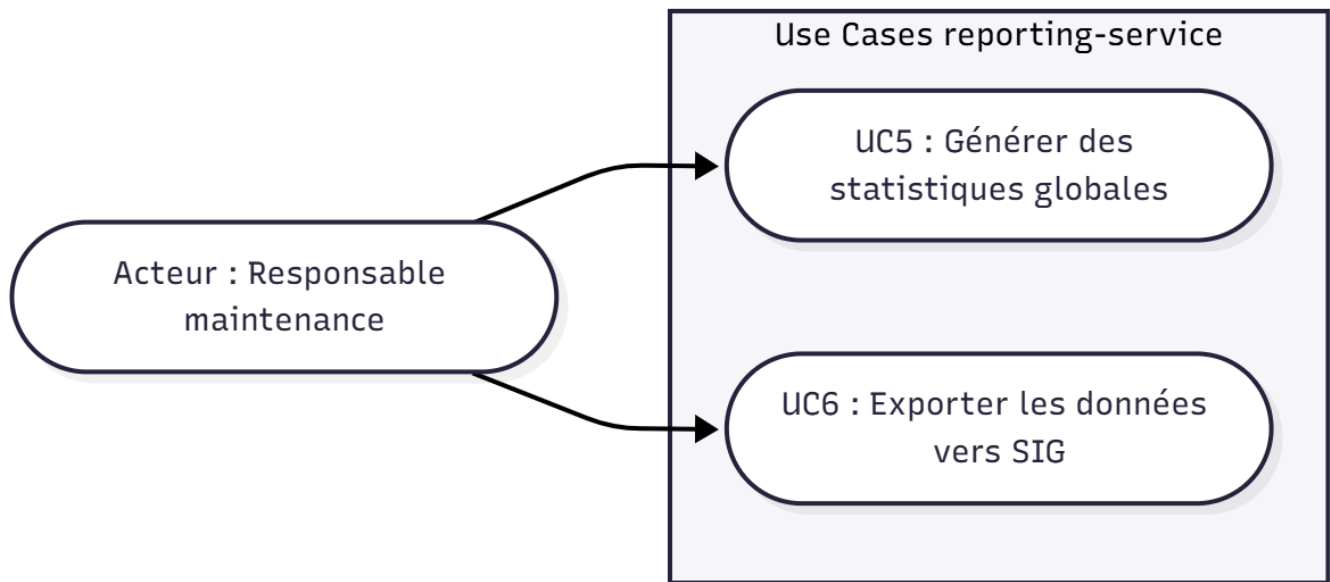
»

reporting-service – Diagramme de classes (si séparé)



- **ReportingController**
 - `getStatistics(filters)`
 - `exportGeoJson(area)`
- **ReportingService**
 - `buildStats(...)`
 - `buildGeoJson(...)`
- **ReportingRepository** (requêtes agrégées sur PostGIS)

Cas d'utilisation typiques :



- UC5: **Générer des statistiques globales** (nombre de défauts par région, par gravité).
- UC6: **Exporter les données vers SIG** (GeoJSON pour QGIS/ArcGIS).